

Agent Workflow Documentation (Enhanced Version)

1. Overview

This document describes the detailed agent workflow implemented in the real estate advisory system. The system integrates Machine Learning (ML), Retrieval-Augmented Generation (RAG), and LangGraph-based agent orchestration to provide intelligent and structured investment recommendations.

2. Workflow Architecture

The system follows a hybrid AI architecture where each stage contributes specific functionality. The workflow ensures both predictive accuracy and contextual reasoning.

- 1 User Input Collection – captures property features
- 2 ML Prediction – predicts price using trained model
- 3 Comparable Analysis – evaluates relative pricing
- 4 RAG Retrieval – fetches contextual market knowledge
- 5 LangGraph Execution – manages workflow states and transitions
- 6 LLM Reasoning – generates structured advisory report
- 7 Fallback Mechanism – ensures reliability during failures
- 8 Output Rendering – displays results in UI

3. Agent State Management

The agent maintains a centralized state object that persists across workflow nodes. This state includes user input, predicted price, comparable summary, retrieved context, reasoning, and final advisory output. This enables consistent and traceable decision-making.

4. LangGraph Nodes (Detailed)

- 1 Retrieve Context Node: Uses vector similarity search to fetch relevant market insights from ChromaDB.
- 2 Analyze Property Node: Compares predicted price with dataset averages and determines valuation signal.
- 3 Generate Report Node: Uses LLM to create structured advisory based on all previous outputs.

5. Data Flow Explanation

The workflow begins with user input which is passed into the ML model for price prediction. The predicted value is then compared with similar properties to understand its relative position. Simultaneously, RAG retrieves supporting knowledge. These components are then combined within the agent workflow to generate a final advisory report.

6. LLM Integration

The system integrates an external LLM through OpenRouter API. The LLM is responsible for converting structured data into human-readable advisory reports. Prompt engineering is used to enforce structured and grounded outputs.

7. Fallback Mechanism

Due to possible API rate limits or failures, a fallback mechanism is implemented. This mechanism generates a rule-based advisory using ML prediction and comparable analysis, ensuring the system remains functional under all conditions.

8. Design Considerations

- 1 Modular architecture for scalability and maintainability
- 2 Separation of ML, RAG, and LLM components
- 3 Use of LangGraph for stateful workflow execution
- 4 Grounding AI outputs using retrieved context
- 5 Robust error handling and fallback strategies

9. Conclusion

The agent workflow demonstrates a practical implementation of a hybrid AI system. By combining prediction, retrieval, and reasoning, the system delivers reliable and structured insights. This architecture can be extended to other decision-support applications.